

DLT Implementation: Challenges

Varga Andrei - 30433

25.05.2026

1 Implementation Difficulties

The hardest part to implement was the NumPy portion. It was not easy for me to understand exactly how NumPy arrays work from the start. The concept that proved most difficult to grasp during implementation was dimensionality. Although initially, as I was already used to from other programming languages, matrices are 2D arrays composed, by a kind of convention, of several $n \times 1$ or $1 \times m$ vectors (with the overall matrix being $n \times m$), when I extracted columns or rows from a matrix I noticed that their shape was not $(1, m)$ or $(n, 1)$ but rather $(m,)$ and $(n,)$, which confused me. I therefore expected to run into problems both with matrix-vector multiplication and with the other methods I needed to use, namely singular value decomposition and QR decomposition. I later discovered that NumPy treats vectors, or 1D arrays, the same way regardless of whether they are intended as column or row vectors, which is different from mathematics and from the functions I used to write in C/C++ for such operations.

2 Conceptual Difficulties

On the conceptual side, I found transforming the system of equations obtained from the point correspondences into the matrix M difficult to understand. The equations that derive the result were also hard to grasp and see as a whole. After that, what was also hard to grasp was SVD. Fortunately, I had paid attention to numerical methods and linear algebra in previous years. The predefined functions helped the implementation process, but I did not want to treat them as a black box, so I went back through online resources to review what these concepts actually meant.

Another thing I found difficult was identifying the practical use of the DLT transform, only to later find out that it is used for calibrating cameras without nonlinear distortions. Another application is 3D reconstruction.